

Portal Session Variables Handling

The Portal Session is a placeholder for all of the variables exchanged between View DFCs and business logic DFCs.

The Portal Session Variables are used to exchange data between different DFCs, such as the View, On Action, On Initialize, and On Load DFCs.

The general rules are:

- ▶ Any External Output returned from the DFCs above or from the controls used in the View Forms is automatically added to the current Portal Session. This happens in the following order:
 - ▶ Before the Screen display – all the External Outputs from the Screen On Initialize or On Load DFC (if set).
 - ▶ After the Screen submit:
 - ▷ First priority – all the Outputs from the View DFC that triggered the submit (triggered Action).
 - ▷ Second priority – all the Outputs from the other Screen Views where the View DFC returned the MergeOutputs Output set to True.
 - ▶ During Action analysis – all the Outputs from the On Action DFC (if set).
- ▶ If a variable already exists in the Portal Session, it is overridden
- ▶ Any External Input required by the above DFCs is taken automatically from the Portal Session without any prefixes (e.g., "Global_", "External_").
- ▶ If such a variable does not exist in the Portal Session and DFC Interface is disabled, Function Interpreter will ask the user for the variable (this is the same behavior that occurs when calling a sub-DFC without passing all the required Inputs) - if DFC Interface is enabled, all the Inputs that are required must be present in the Portal Session

Choosing a variable name according to a special pattern can affect its behavior in Process Builder. The pattern is: [Global_ | External_] variable name [_UI]. The optional prefix (global or external) defines the variable scope:

Variable Scope	Value Merged to Session	Value Restored Upon Back	Value Passed to Sub-Portal	Can Be Used as PortalExecute Input
Normal (no prefix)	Yes	Yes	No	No
External	Yes	Yes	Yes	Yes
Global	Yes	No	Yes	Yes

The optional suffix (UserInput) defines at which moment the variable's values snapshot is pushed to the call stack:

- ▶ Normal (no suffix) – the variable values as they were before the display of the Screen are used for the snapshot
- ▶ UserInput – the variable values after the Screen is submitted (e.g., the value from a User Input) are used for the snapshot

These are some examples:

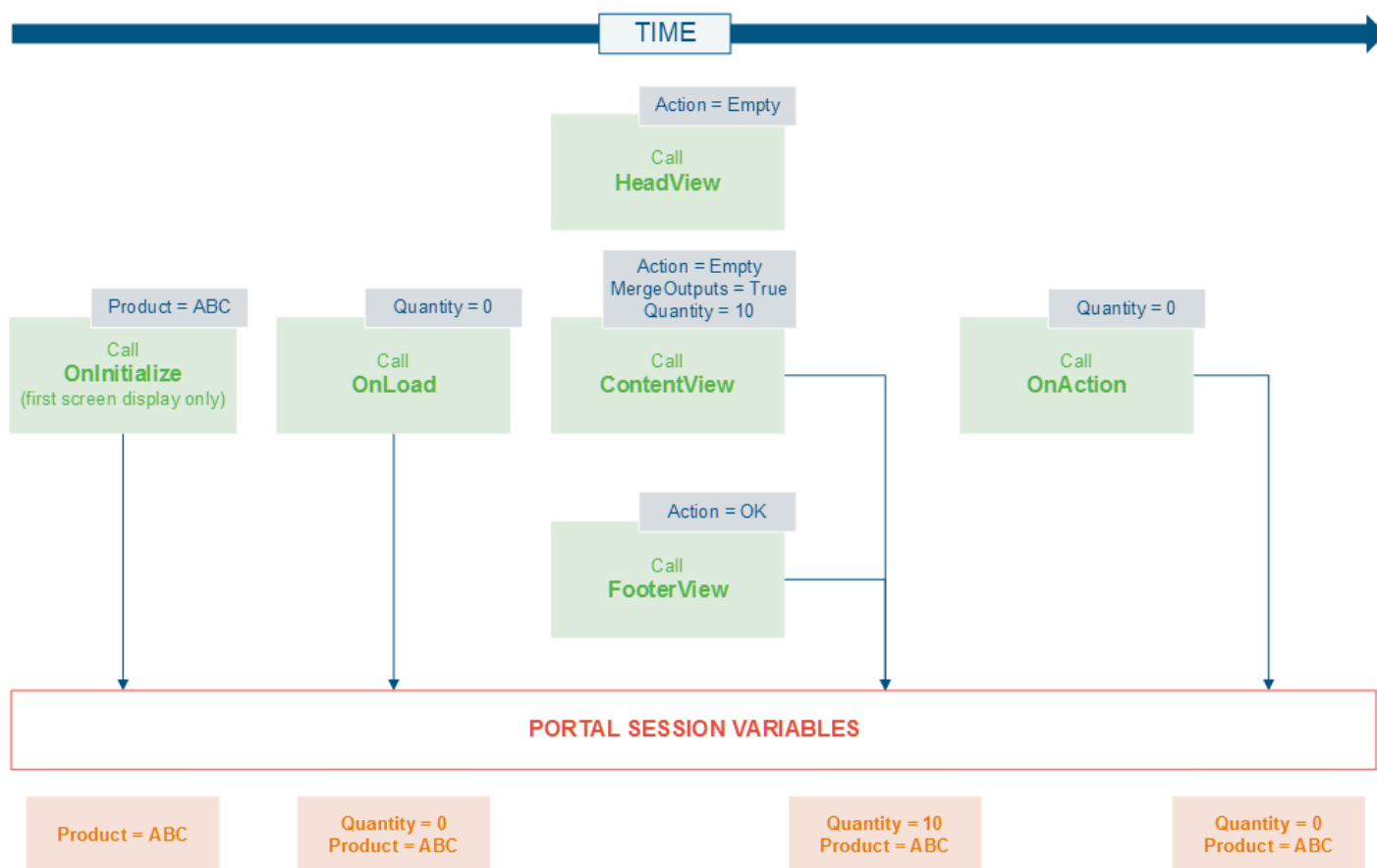
- ▶ Container – defines a normal variable that will be:
 - ▷ Pushed to a stack before the Screen display
 - ▷ Restored upon back (to the value before the Screen display)
 - ▷ Not passed to Sub-Portals

- ▶ **Container_UI** – defines a User Input variable that will be:
 - ▷ Pushed to the stack after the Screen submit (with values from the user)
 - ▷ Restored upon back (to the value originally entered by the user)
 - ▷ Not passed to Sub-Portals
- ▶ **External_Container_UI**
 - ▷ Pushed to the stack after the Screen submit (with values from the user)
 - ▷ Restored upon back (to the value originally entered by the user)
 - ▷ Passed to Sub-Portals

The suffix “_UI” is automatically removed when adding a variable to the Session Variables.

Event Order and Session Variables

The flow below illustrates the chain of events during a Screen display and a user Action together with the state of the Portal Session Variables at any given time. Imagine that this particular Screen has three Views (as presented below) and the On Initialize Screen is set. The user presses the OK button on FooterView, which triggers the On Action DFC:



Immediate Variables

Immediate variables are Portal Session Variables that are not merged into the Portal Session, as they are visible only within the scope of a single Screen.

Using immediate variables is an ideal solution for scenarios where the variable provided on the View as the User Input is required only for an On Action DFC.

Immediate variables will be passed into the On Action DFC, but they will not be kept in the Portal Session. Accordingly, they will not pollute the session with variables that were needed only for immediate processing.

The External Output of the View is considered an immediate variable, as it starts with the "_" prefix (e.g., _Container). The "_" prefix is automatically removed when passing immediate variables into the On Action DFC. Therefore, your On Action DFCs can be designed for both normal and immediate variables in the same way.

As a best practice, consider using immediate variables as a starting point. Change them into regular variables only if they are needed on another screen. This helps to keep the Portal Session small.

Immediate variables can be also used in Forms.

Screen Instance Variables

When you need to store some Screen-specific information in a Portal session, you can use the special Screen Instance Variable (e.g., to store the last used grid profile on each Screen). To define the Screen Instance Variable, prepend the variable name with the /Screen prefix (so that the variable name is /Screen/NameOfTheVariable).

You can use this kind of variable only on corresponding Screens, and these are removed from the Portal Session when there is no Screen instance using them in the Screen stack.

For example: after defining the /Screen/GridProfile variable, it will be included in the Portal Session and it will not overwrite variables of the same name from different Screens. You can then reuse the existing variable (without the prefix) on different Screens, which will result in different profiles on each Screen.

Debug Information

The Debug Information is supported only in the Screen Flows server mode.

During development, it is important to know at any time what current variables are available in the Portal Session. When using Process Builder Test Run on a Screen, this information is displayed in the bottom-right corner of the Screen. After hovering over the white rectangle, the debug info is displayed:

```
DEBUG (SVL.I.055)EXIT
```

Screen	Instance	Layout	OnInitialize	OnLoad
RCO	RCO	WMS_CONSOLE	--	--

RECEIVING

EmployeeFacility	C1P1	String
Html		String
OrderNo		String
OrderType	0	String
OrderTypeList	16,17 (2)	Object[]
ScreenTitle	Receiving Orders	String

t = 601 ms.

- ▶ **Screen** – the current Screen code
- ▶ **Instance** – the Screen instance
- ▶ **Layout** – the Layout of the Screen
- ▶ **On Initialize** – the DFC called On Initialize
- ▶ **On Load** – the DFC called On Load
- ▶ **Action** – the last Action (in the case of an Action chain, all the Actions and On Action DFCs are displayed)
- ▶ A list of all the variables along with their current values and types
- ▶ The last refresh time (in milliseconds) – this includes Process Builder and business logic processing

Advanced Test Run

If the Screen requires multiple variables to work properly, you can use [Advanced Test Run](#) to be able to easily test if everything works as it should. Upon entering the Test Run, you will be prompted to specify the variables values that will be used to initialize the Portal Session. You can even use whole value sets and copy them to/from the clipboard (in the JSON format).

Screen Initialization

In many cases, when displaying a Screen for the very first time, some variables may be required that are not yet available in the Portal Session. In such a scenario, you should use the [Screen: Parameters Tab](#). If setting parameters is not enough, you can use the On Initialize or On Load DFC, which will be called before displaying the new Screen (for more information on the On Initialize DFC, see [Screen: General Tab](#)).

These are the External Inputs available:

- ▶ **Screen** – the screen code of the screen being initialized, the mandatory Input
- ▶ Any Portal Session Variable

The External Outputs are all of the variables to be merged to the Portal session.

The PortalScreen_Initialize DFC template is delivered.

The On Initialize DFC is called only once, when the Screen is displayed for the very first time. The system analyzes the call stack to check if the Screen instance had already been displayed in the past. The On Load DFC is called every time the Screen is displayed.

The screen with the navigation type set to Home will clear the call stack so that next time the On Initialize DFC is called.